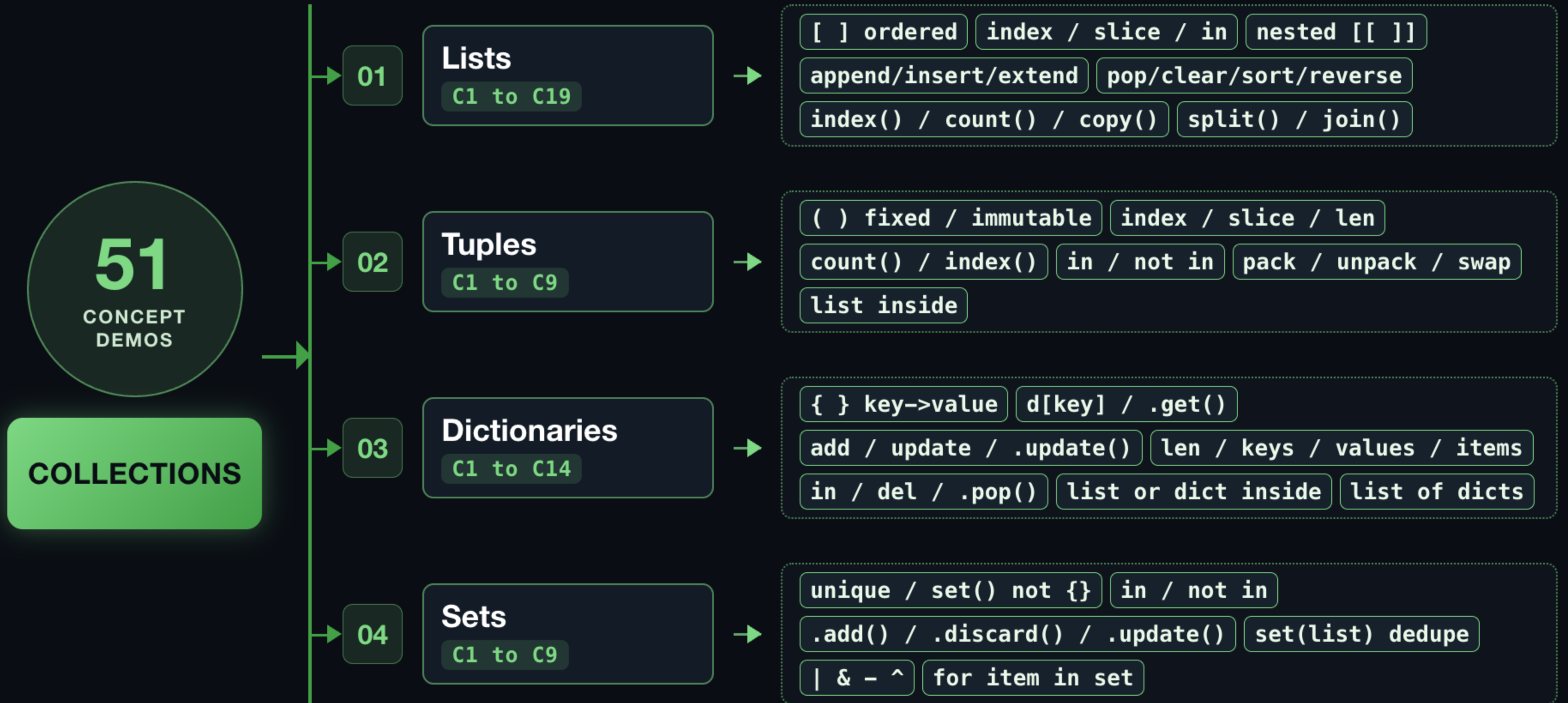


Section 05: Collections Concept Map

The full path through this section - we open here before each concept, then dive in.



Collections: **Lists** - Ordered and Changeable (1 of 3)

A list is an ordered, changeable collection that holds many values in one place, written inside square brackets.

#	CONCEPT	SAMPLE CODE
1	Create a list <code>[]</code> Hold many values in order, in square brackets.	<code>names = ["Sam", "Tom", "Ben"]</code>
2	Access by index <code>[i]</code> Read an item by position; 0 is first, -1 is last.	<code>print(names[0])</code>
3	Change a list Add, replace, or remove items in place.	<code>names.append("Joe")</code>
4	Sort in place <code>.sort()</code> Arrange items from low to high, in place.	<code>scores.sort()</code>
5	String to list <code>.split()</code> Cut a string into a list at a separator.	<code>date_text.split("-")</code>
6	List to string <code>.join()</code> Join list items into one string with a separator.	<code>" ".join(words)</code>
7	Insert at index <code>.insert()</code> Put an item at a position, shifting the rest right.	<code>names.insert(1, "Amy")</code>

Collections: **Lists** - Ordered and Changeable (2 of 3)

A list is an ordered, changeable collection that holds many values in one place, written inside square brackets.

#	CONCEPT	SAMPLE CODE
8	Add many items <code>.extend()</code> Add every item from another list at once.	<code>names.extend(["Ben", "Joe"])</code>
9	Remove and return <code>.pop()</code> Remove an item and get it back.	<code>remove_ben = names.pop()</code>
10	Empty the list <code>.clear()</code> Remove everything, leaving an empty list.	<code>names.clear()</code>
11	Reverse order <code>.reverse()</code> Flip the list order in place.	<code>names.reverse()</code>
12	Find position <code>.index()</code> Get the index where a value sits.	<code>names.index("Tom")</code>
13	Count a value <code>.count()</code> Count how many times a value appears.	<code>letters.count("a")</code>

Collections: **Lists** - Ordered and Changeable (3 of 3)

A list is an ordered, changeable collection that holds many values in one place, written inside square brackets.

#	CONCEPT	SAMPLE CODE
14	Copy a list <code>.copy()</code> Make a separate list; a plain = only aliases.	<pre>separate = original.copy()</pre>
15	Index and slice <code>[a:b:c]</code> start:stop:step, exactly like slicing strings.	<pre>print(nums[1:3])</pre>
16	Membership test <code>in</code> Check whether a value is in the list.	<pre>"Sam" in ["Sam", "Tom"]</pre>
17	Not in test <code>not in</code> Check whether a value is not in the list.	<pre>"Zoe" not in ["Sam", "Tom"]</pre>
18	Nested lists <code>[i][j]</code> A list inside a list; two indexes reach inside.	<pre>print(grid[0][2])</pre>
19	Nested with strings Group lists together and pick one item.	<pre>print(groceries[1][2])</pre>

Collections: Split and Join

The bridge between text and lists: `split()` turns a string into a list, `join()` turns a list back into a string.

`split()`

A string becomes a list, cut at the separator: `"a-b".split("-")`.

`join()`

A list becomes a string, glued by a separator: `".join(words)"`.

Opposites

`split()` goes text to list; `join()` goes list back to text.

SPLIT AND JOIN

```
"2026-06-17".split("-")  
# ['2026', '06', '17']
```

```
".join(["Python", "is", "fun"])  
# Python is fun
```

No argument? `split()` cuts on any whitespace.

Collections: Tuples - Fixed and Unpackable (1 of 2)

A tuple is an ordered group of values written in round brackets that cannot change after you create it.

#	CONCEPT	SAMPLE CODE
1	Create a tuple <code>()</code> Group values that belong together in round brackets.	<pre>player = ("Sam", 25)</pre>
2	Immutable, cannot change Changing a tuple item is an error, so it stays safe.	<pre>player[0] = "Tom" # TypeError</pre>
3	Two methods <code>.count()</code> <code>.index()</code> <code>count()</code> how many times a value appears; <code>index()</code> where.	<pre>print(scores.count(20))</pre>
4	Slice a tuple <code>[a:b:c]</code> <code>start:stop:step</code> ; a slice always gives a tuple.	<pre>print(scores[0:3])</pre>
5	Membership test <code>in</code> <code>not in</code> Check whether a value is in the tuple (read-only).	<pre>print(20 in scores)</pre>

Collections: Tuples - Fixed and Unpackable (2 of 2)

A tuple is an ordered group of values written in round brackets that cannot change after you create it.

#	CONCEPT	SAMPLE CODE
6	Commas make it <code>,</code> Commas build the tuple; brackets are optional.	<code>player = "Sam", 25</code>
7	Unpack a tuple Give each item its own name in one line.	<code>name, age, city = player</code>
8	Swap in one line Swap two variables, then save the result as a new tuple.	<code>a, b = b, a</code>
9	Mutable list inside Slots are fixed, but a list inside a slot can still change.	<code>player[1].append(100)</code>

Collections: Dictionaries (1 of 2)

A **dictionary** stores each value under a named **key**, so you look things up by a label instead of a position.

#	CONCEPT	SAMPLE CODE
1	Create a dictionary <code>{}</code> Keys label each value instead of a position.	<pre>student = {"name": "Sam", "age": 20}</pre>
2	Look up by key <code>[key]</code> Square brackets read the value for a key.	<pre>print(student["name"])</pre>
3	Safe lookup <code>.get()</code> Returns None or a default, never a crash.	<pre>print(student.get("email", "n/a"))</pre>
4	Add or update a pair Assign to a key to add it or replace it.	<pre>student["email"] = "sam@example.com"</pre>
5	Set many pairs <code>.update()</code> Merge several pairs from another dict at once.	<pre>student.update({"age": 21, "city": "Mumbai"})</pre>
6	Build from empty <code>{}</code> Start with <code>{}</code> and add pairs one by one.	<pre>student = {}</pre>
7	Count the pairs <code>len()</code> <code>len()</code> returns how many pairs are stored.	<pre>print(len(student))</pre>

Collections: Dictionaries (2 of 2)

A **dictionary** stores each value under a named **key**, so you look things up by a label instead of a position.

#	CONCEPT	SAMPLE CODE
8	All keys and values <code>.keys()</code> <code>.values()</code> See every label or every value at once.	<pre>print(list(student.keys()))</pre>
9	All key-value pairs <code>.items()</code> Get each pair as a (key, value) tuple.	<pre>print(list(student.items()))</pre>
10	Is a key present? <code>in</code> <code>in</code> and <code>not in</code> check keys, not values.	<pre>print("name" in student)</pre>
11	Remove a key <code>del</code> <code>.pop()</code> <code>del</code> drops a pair; <code>.pop()</code> returns its value.	<pre>old_age = student.pop("age")</pre>
12	Value is a list A value can be a list you index or grow.	<pre>student["scores"].append(100)</pre>
13	Nested dictionary A value can be another dict you chain into.	<pre>print(student["address"]["city"])</pre>
14	List of dictionaries Many records with the same labels, like rows.	<pre>print(people[0]["name"])</pre>

Collections: **Sets** for Unique Values (1 of 2)

A set is an unordered collection of unique values, so it never stores duplicates and has no index to reach items by position.

#	CONCEPT	SAMPLE CODE
1	Set of unique values <code>{ }</code> Curly braces hold unique, unordered values.	<code>sam_skills = {"python", "ml", "data"}</code>
2	Empty set <code>set()</code> Use <code>set()</code> ; <code>{ }</code> makes an empty dict instead.	<code>empty = set()</code>
3	Membership test <code>in</code> <code>not in</code> Fast check if a value is in the set.	<code>print("ml" in sam_skills)</code>
4	Add one value <code>.add()</code> Grows the set; a duplicate add does nothing.	<code>sam_skills.add("sql")</code>

Collections: Sets for Unique Values (2 of 2)

A set is an unordered collection of unique values, so it never stores duplicates and has no index to reach items by position.

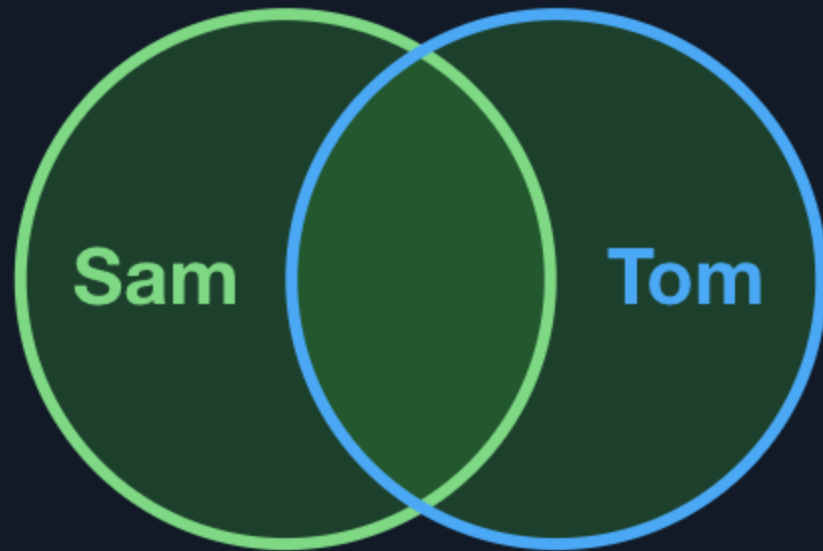
#	CONCEPT	SAMPLE CODE
5	Add many values <code>.update()</code> Bulk-adds items from a list or another set.	<code>sam_skills.update(["sql", "go"])</code>
6	Remove a value <code>.discard()</code> <code>.remove()</code> discard is safe; remove raises <code>KeyError</code> .	<code>sam_skills.discard("java")</code>
7	Set math <code> </code> <code>&</code> <code>-</code> Union, intersection, and difference.	<code>sam_skills tom_skills</code>
8	Symmetric difference <code>^</code> Values in exactly one set, not both.	<code>sam_skills ^ tom_skills</code>
9	Reach the items <code>for ... in</code> No index on a set, so loop over it (Section 06).	<code>for skill in sorted(sam_skills):</code>

Collections: Set Math - Combine Two Sets

Sam's and Tom's skills, combined three ways. The shaded area is the result; we print it with `sorted()`.

Sam = {python, ml, data} | **Tom** = {python, java, sql} | shared: **python**

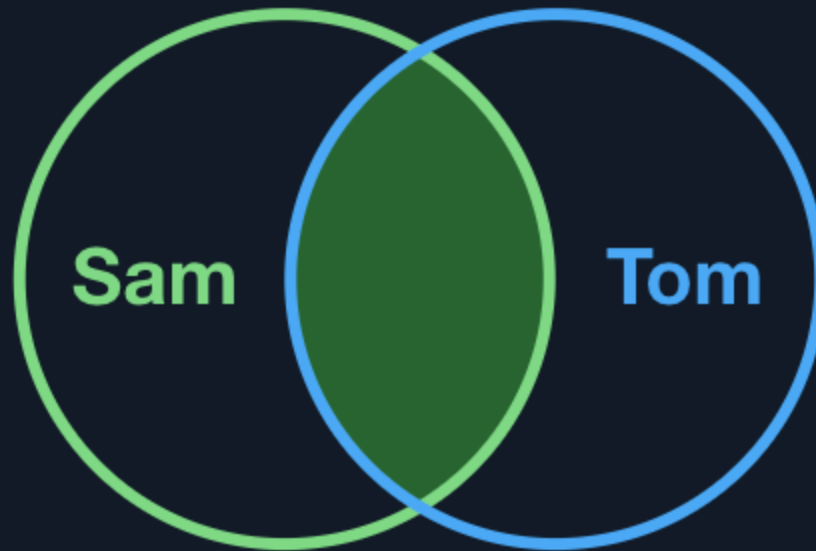
Union |



{data, java, ml,
python, sql}

everything in EITHER set

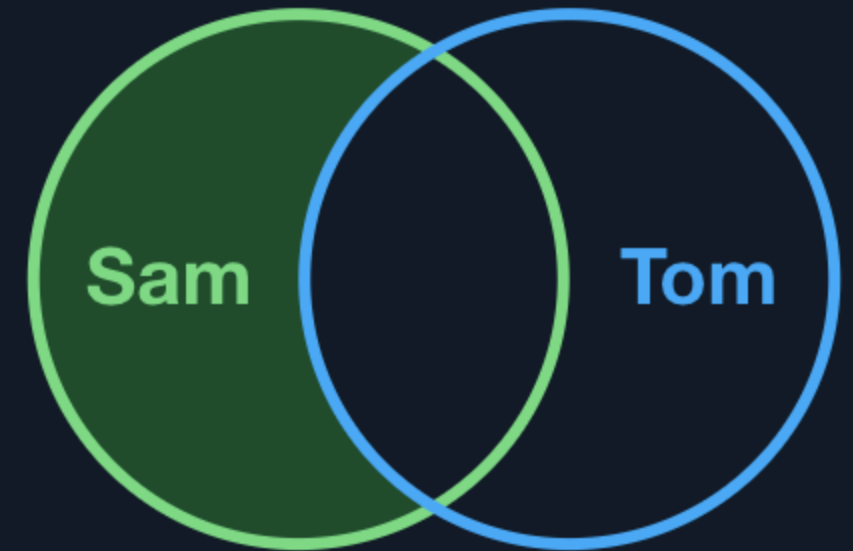
Intersection &



{python}

only what is in BOTH sets

Difference -



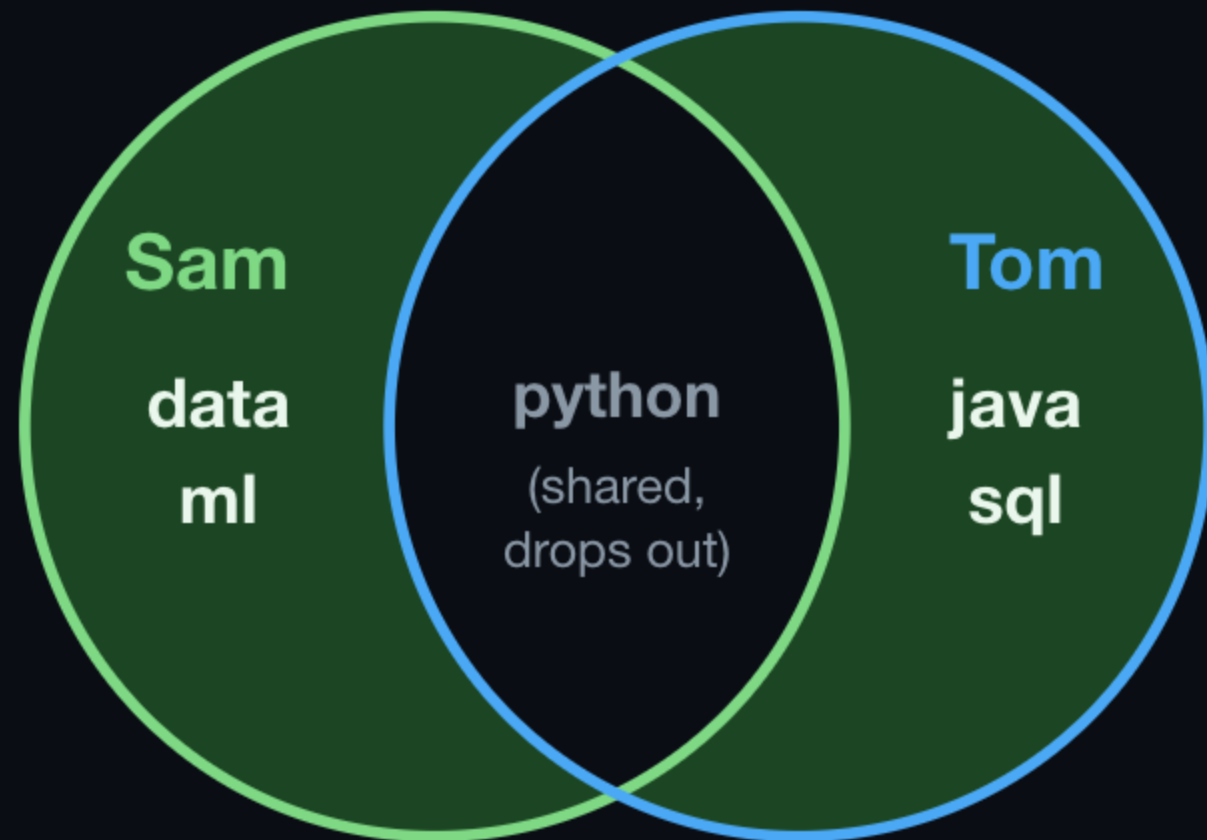
{data, ml}

in Sam, but NOT Tom
(Sam - Tom)

Collections: Symmetric Difference - **in Exactly One Set**

The two skills each person has ALONE - everything except what they share.

Sam = {python, ml, data} | **Tom** = {python, java, sql}



Sam ^ Tom

{data, java, ml, sql}

Keep everything that is in **exactly one** set.

python is in BOTH, so it is the ONE value that drops out.